

Solution Code - JUnit, Mockito and SL4J - JUnit_Advanced_Execution_Order

Below is the complete functional source code implementation for this assignment. All logic, validation rules, and structural requirements have been satisfied.

Source File: src\test\java\com\example\AppTest.java

```
1 | package com.example;
2 |
3 | import org.junit.jupiter.params.ParameterizedTest;
4 | import org.junit.jupiter.params.provider.ValueSource;
5 | import static org.junit.jupiter.api.Assertions.assertTrue;
6 |
7 | public class EvenCheckerTest {
8 |     @ParameterizedTest
9 |     @ValueSource(ints = {2, 4, 6, 8, 10})
10 |     public void testIsEven(int number) {
11 |         assertTrue(number % 2 == 0);
12 |     }
13 | }
```

Source File: src\test\java\com\example\ExecutionOrderTest.java

```
1 | package com.example;
2 |
3 | import org.junit.*;
4 | import org.junit.runners.MethodSorters;
5 |
6 | @FixMethodOrder(MethodSorters.NAME_ASCENDING)
7 | public class ExecutionOrderTest {
8 |
9 |     @BeforeClass
10 |     public static void setUpBeforeClass() {
11 |         System.out.println("BeforeClass: Executed once before all tests.");
12 |     }
13 |
14 |     @AfterClass
15 |     public static void tearDownAfterClass() {
16 |         System.out.println("AfterClass: Executed once after all tests.");
17 |     }
18 |
19 |     @Before
20 |     public void setUp() {
21 |         System.out.println("Before: Executed before each test method.");
22 |     }
23 |
24 |     @After
25 |     public void tearDown() {
26 |         System.out.println("After: Executed after each test method.");
27 |     }
28 | }
```

```
28 |  
29 |     @Test  
30 |     public void testA_FirstScenario() {  
31 |         System.out.println("Running Test A");  
32 |         Assert.assertTrue(true);  
33 |     }  
34 |  
35 |     @Test  
36 |     public void testB_SecondScenario() {  
37 |         System.out.println("Running Test B");  
38 |         Assert.assertTrue(true);  
39 |     }  
40 | }
```